



SDAIA ACADEMY · APPLIED GENERATIVE AI

From token to typed output

Day 1 · Sunday 2 August 2026 · Musa Ibn Rashid

The model is a function. Today we make it a reliable one.

PRESENTER NOTES

Good morning, and welcome. Over the next five days we are going to build generative AI that actually ships, not slides about generative AI.

Today has one destination: by half past two you will have written a function that sends a prompt and gets back JSON that your own code can branch on.

That is the moment this stops being a chat toy and becomes a component in a system, and everything for the rest of the week sits on top of it.



YOUR INSTRUCTOR

About me

Musa Ibn Rashid — Software Engineering from KFUPM, QA Engineering Intern at Sawt, Qimam Fellow, and KAUST AI Summer Internship alumni.

AI Instructor at SDAIA Academy, specialising in Applied Generative AI, Computer Vision, and Prompt Engineering. I like helping people ship, empowering tech talent, and building AI that bridges complex ideas with practical execution.

For fun I created **Ice ❄️ Offering**.

All my life I've been travelling.

أحب اسجل صوتيات كثيرة واطول.

I literally fall asleep wherever I am if I am sleepy.

PRESENTER NOTES

Keep this to two minutes. The facts are there so they remember you as a person, not just the syllabus.

Ice Offering is a good laugh if someone asks — don't dwell on it unless the room wants to.



YOUR TURN

Quick intro — everyone, thirty seconds

NAME

First and last — how you want to be called this week.

UNIVERSITY

Where you studied or where you are studying now.

MAJOR

Your field — engineering, CS, business, anything.

YEAR

Graduated when, or what year you are in if still a student.

One sentence each. We will go around the room
— no speeches.

PRESENTER NOTES

Start on the left, work around. If someone freezes, prompt them with "name and major is enough."

You are listening for who has coded before and who has not — that is your pairing data for the project.

STAY CONNECTED

Join the WhatsApp group

**Scan the QR code on screen**

I will open it now — announcements, links, and quick help between blocks.

Why bother

- Colab links and slide URLs when you need them on your phone.
- Ask a question without stopping the whole room.
- Pair up for the project if you miss someone on day one.

No confidential data in the group. Treat it like a classroom channel.

PRESENTER NOTES

Have the QR ready before you show this slide. Give them ninety seconds to scan and confirm they are in.

If someone cannot use WhatsApp, tell them everything also lives on the course site.



THE WEEK



Where today sits

SUNDAY

From token to typed output. The model as a reliable function.

MONDAY

Retrieval you can measure — RAG, hybrid search, and a golden set that scores it.

TUESDAY

Tools and agents — the model requests, your code executes. Yesterday's retriever becomes a tool.

WEDNESDAY

Production and the numbers — latency, caching, and what this costs per user per month.

THURSDAY

Break it, then defend it — prompt injection, red teaming, and your demo.

Each day ends higher than it started. If Thursday feels reachable today, tell me.

There is no such thing as a dumb question. If you are wondering, someone else is too — ask.

PRESENTER NOTES

Notice that every day builds on the one before it. Monday's retriever becomes Tuesday's tool, and Tuesday's agent is what you instrument on Wednesday and attack on Thursday.

That means falling behind on any one day is expensive, so please tell me the moment you are stuck rather than at the end of the block.

Thursday afternoon you present to the room, in pairs, for four minutes.

Say the dumb-question line slowly and mean it — it sets the room for the whole week.



EVERY DAY



How the hours run

We announce 9:00. We start at 9:15. The buffer is for keys, laptops, and coffee.

9:00 – 10:00	Block 1	60 min
10:00 – 10:20	Break	20 min
10:20 – 11:00	Block 2	40 min
11:00 – 11:20	Break	20 min
11:20 – 12:00	Block 3	40 min
12:00 – 1:00	Lunch	60 min
1:00 – 1:50	Block 4	50 min
1:50 – 2:00	Break	10 min
2:00 – 2:30	Block 5	30 min

PRESENTER NOTES

Same grid every day — once they know it, they stop asking when lunch is.

Never announce the real start time as nine. Nine fifteen is when teaching begins; nine to nine fifteen is setup.

TWENTY MINUTES · NOTHING TO DO WITH AI

Two Truths and a Lie

1

Write three statements about yourself — two true, one false. Make the lie *boring enough to believe*.

2

Groups of five. Read yours out; the group votes on the lie; each group picks their best faker.

3

The final — group winners to the front. Whole room votes by raised hand. Small prize.

4

I go last, out of competition. Then: name + **one sentence** on what you want AI to do for you by Thursday.

Remember your group of five — those become your project team pool on Monday.

PRESENTER NOTES

Three minutes to write. Ten minutes in heats. Six minutes for the final. One minute wrap.

The boring-lie instruction is what makes this work — say it twice before they start writing.

Capture the Thursday AI wishes on a flipchart. That flipchart is the project idea pool.

You go last, out of competition, after they have all done it — lands better than opening with yourself.

FIFTEEN MINUTES · IN YOUR BROWSER

Pre-test

Twenty questions across the whole week — mechanics, retrieval, agents, cost, security. Most of it we have not taught yet.

This measures me, not you. Zero is a fine score this morning. On Thursday you take the identical test and we look at the difference.

How to take it

- Open the course site → **Start pre-test**.
- No timer. Answer what you can, guess the rest.
- Nothing is shared — it saves to your own browser.
- Hit **Copy my result** at the end and read it out.

Open the pre-test →

PRESENTER NOTES

Frame this before anyone opens it, or half the room will feel tested on material they have never seen.

Say it plainly: this measures the course, not them, and a low score right now is exactly what the week is for.

Fifteen minutes is generous. When most people are done, move on rather than waiting for the last person.

Collect the scores for the SDAIA sheet — the copy button puts it on their clipboard in one tap.

DAY 1

What you will be able to do by 2:30

- 01** Explain what a token is, and why the same sentence costs more in Arabic than in English.
- 02** Call a generative model from Python and read the token counts off the response.
- 03** Turn temperature and a system instruction and predict what will change.
- 04** Force the model to return JSON that matches a schema you defined.
- 05** Wrap all of it in one reusable `ask()` function with retries.

PRESENTER NOTES

I want you to read these five as promises, not as a wish list. At half past two I am going to put this slide back up and ask you which ones you can do.

Number four is the one that matters most, and it is the one nobody teaches. Almost every course stops at number three.

If you finish early, help the person next to you — I have paired coders with non-coders deliberately.



PART ONE

How the thing actually works

Forty minutes of mechanics. Everything after this depends on it.

PRESENTER NOTES

We are going to spend the next forty minutes under the bonnet, and some of you will find the first ten minutes of it easy.

Stay with me anyway, because the vocabulary we set here is what I will use for the rest of the week without re-explaining it.

Two kinds of model

Discriminative

Draws a boundary. Given input, pick a label.

"Is this transaction fraud?" → **yes / no**

Fixed set of answers. You can measure accuracy directly.

Generative

Models the distribution. Given input, produce new content.

"Write the fraud alert for this customer." → **a paragraph that never existed**

Infinite possible answers. "Correct" is much harder to define.

That second column is why everything this week is about **constraining** the output.

PRESENTER NOTES

The practical difference is not the architecture, it is what you can promise your users. A classifier has a fixed set of answers, so you can test it exhaustively.

A generative model can return anything, which is exactly why the rest of today is about narrowing what it is allowed to say.

That is one slide on theory. We are moving on.

MECHANICS

A token is not a word

The model never sees letters or words. Text is cut into **tokens** — common chunks, learned from data — and each one becomes a number.

"Generative models are unbelievably useful."

Gener ative models are un believ ably useful .

6 words · **10 tokens**. Common words are one token;
rare ones get broken up.

You are billed per token, in and out. Tokens are the unit of everything: cost, limits, speed.

PRESENTER NOTES

Look at "unbelievably". It is one word to you and four pieces to the model, because it is rare enough that the tokenizer never learned it whole.

The rule of thumb for English is about four characters per token, so a hundred words is roughly a hundred and thirty tokens.

Hold on to that number, because in two slides it is going to be very different for Arabic.

MECHANICS · LIVE

The same sentence costs more in Arabic

English

"The National Data Governance policy applies to every entity."

≈ **11 tokens**

Arabic — the same meaning

تنطبق سياسة حوكمة البيانات الوطنية على كل جهة.

≈ **26 tokens**

Type here — approximate token count

تنطبق سياسة حوكمة البيانات الوطنية على كل جهة.

20 tokens · 46 characters

Approximate. A heuristic, not the real tokenizer: ~4 characters per token for Latin script, ~2 for Arabic.

PRESENTER NOTES

Two to three times the tokens for the same sentence. That is not a rounding error, it is your budget, and nobody warns you about it.

Type something of your own into the box on the right and watch the count. Then paste the Arabic translation of it underneath and watch it jump.

This comes back on Wednesday when we price an internal assistant, and the Arabic version of the same product costs noticeably more to run.

MECHANICS

The context window

Everything the model can see at once: your system instruction, the conversation so far, any documents you pasted in, *and* the answer it is writing.

Measured in tokens. Go over it and the call fails, or the earliest content silently falls out.

This is why chunking exists tomorrow.

The consequences

- You cannot paste a 400-page policy manual into a prompt.
- A long chat quietly forgets its own beginning.
- Bigger windows are real, but you pay for every token you fill.

PRESENTER NOTES

The window is not memory. Nothing is remembered between calls — every request re-sends the entire conversation, which is why long chats get expensive.

Say the last line slowly, because tomorrow morning somebody will ask why we bother cutting documents up, and this slide is the answer.

MECHANICS

It is predicting the next token. That is the whole trick.

"The capital of Saudi Arabia is"

Riyadh – 94% the – 2% located – 1% a – 0.4%

It picks one, appends it, and asks the same question again. Hundreds of times.

Nothing in that loop checks whether the sentence is **true**. It checks whether it is **likely**.

Fluent and wrong are not opposites here. Fluency is the objective; truth never was.

PRESENTER NOTES

This is the single most useful thing to understand about these models, and it explains almost every failure you will hit this week.

There is no lookup, no database, no fact check in that loop. It is picking a likely next piece of text, over and over.

When you understand that, hallucination stops being surprising and starts being something you design around.

THE DIAL YOU ACTUALLY TURN

Temperature

Same prompt three times: "Write one line describing a data governance policy."

temperature = 0.0

"A data governance policy defines how data is managed."

Run it three times, get this three times.
Predictable.

temperature = 0.7

"A data governance policy sets the rules for how an organisation handles its data."

Varied wording, same substance.

temperature = 1.2

"Think of governance as the traffic law of your data highways — signs, lanes, and fines."

Creative. Also where it starts inventing.

Extraction, classification, anything you parse: **0.0 to 0.2**. Drafting and ideation: 0.7 up.

PRESENTER NOTES

Temperature reshapes the probability list before it samples. Near zero it always takes the most likely token, which is why the output repeats exactly.

The mistake I see constantly is people running extraction tasks at the default temperature and then complaining that the output is inconsistent.

In the lab you will run the same prompt three times at zero and three times at one point two, and see this for yourself.

MECHANICS

Hallucination is a property, not a bug

The model produces likely text. When it has no grounding, likely text still arrives — confident, well formatted, and wrong.

It will invent a policy number, a citation, an article, a date. Fluently.

There is no patch coming. Every mitigation on this slide is architecture, not a setting.

You design around it

- **Ground it** — give it the document. That is tomorrow.
- **Cite it** — make it show which chunk it used.
- **Type it** — a schema removes room to waffle. That is this afternoon.
- **Check it** — validate in code before you act.

PRESENTER NOTES

I want to be honest with you here: this is not a defect that gets fixed in the next model release. It is what next-token prediction does when it lacks grounding. So we do not try to stop it. We build systems where a made-up answer is caught, or cannot be acted on, or is visibly unsupported by a citation. Every one of those four bullets is a day of this course.

THE SPINE OF THE WEEK

Four ways to make a model useful

APPROACH	WHAT IT IS	COST	WHEN
Prompting	Better instructions, examples, output format	Minutes. Free.	Always first. Try this before anything else.
RAG	Retrieve your documents, put them in the prompt	Days. Low.	The model needs knowledge it does not have.
Tools	The model requests, your code executes	Days. Low.	It needs live data, or needs to <i>do</i> something.
Fine-tuning	Retrain weights on your examples	Weeks. High.	You need a style or format that prompting cannot hold.

Most people reach for fine-tuning and should have reached for RAG.

PRESENTER NOTES

This is the most useful table in the whole course and I will point back at it every single day.

Read the cost column across. Prompting is minutes, retrieval is days, fine-tuning is weeks and needs data you probably do not have.

Fine-tuning teaches a model how to say things. It does not teach it new facts, and that is the confusion that burns budgets.



PART TWO

Make it reliable

Same model, better instructions. This is the cheapest win you will ever get.

PRESENTER NOTES

Everything so far has been how the model works. From here it is what you do about it.

And we start with prompting, because it costs nothing and it is where most of the improvement actually lives.

ROLES

System instruction vs user message

System

Who the model is, what it may do, how it answers.
Set by **you**, the developer. Sent on every call.

"You are an HR policy assistant. Answer only from the provided documents. If the answer is not there, say so."

User

The question. Comes from **outside** — possibly from someone who does not wish you well.

"How many annual leave days do I get?"

That separation is a **security boundary** — and on Thursday you will spend the morning attacking it.

PRESENTER NOTES

Everything on the left is your policy. Everything on the right is untrusted input, in exactly the way a web form is untrusted input.

The uncomfortable part, and we will prove it on Thursday, is that both of these end up as text in the same window, so the boundary is a convention rather than a wall.

Remember this slide. I will put it back up on day five before we break it.

PROMPTING

Anatomy of a prompt that holds

Role	Who is answering, and what they are responsible for.
Context	The documents, the data, the conversation so far — clearly delimited.
Task	One instruction, in the imperative. Not three questions in a paragraph.
Format	Exactly what shape the answer takes. JSON, a table, three bullets, under 50 words.
Examples	One or two done correctly. Worth more than another paragraph of instruction.

Delimit your context: `<document> ...`
`</document>`. Vague prompts fail in ways you
cannot debug.

PRESENTER NOTES

When a prompt misbehaves, walk these five and find the missing one. Nine times out of ten it is format, and the tenth time it is that you asked three things at once.

The delimiters matter more than they look. They tell the model where your instructions end and untrusted content begins, which is also a first line of defence on Thursday.

PROMPTING

Show it, don't tell it

Before — zero-shot

"Classify this support ticket."

→ "This appears to be a complaint about slow service, possibly urgent."

A different shape every time. Unparseable.

After — two examples

"Ticket: 'Portal is down.' → outage | high

Ticket: 'How do I reset my password?' → account | low

Ticket: '...' → "

→ "billing | medium"

Same shape every time. Now you can split on the pipe.

Two examples usually beat two paragraphs of instruction. Cheaper, too.

PRESENTER NOTES

Examples do something instructions cannot: they demonstrate the output shape, the level of detail and the vocabulary all at once.

Two or three is normally enough. If you find yourself at ten examples, that is the signal that the task might genuinely need fine-tuning.

And notice what the right-hand column bought us — output we can parse, which is exactly where we are going after the break.



PART THREE · THE CEILING

Make it typed

The difference between a chatbot and a component in a system.

PRESENTER NOTES

This last half hour is the part of today that earns the word "applied", and it is missing from almost every generative AI course I have seen. Everything up to now made the model behave better. What follows makes its output something software can depend on.

THE CEILING

Free text is unusable inside software

You ask for the customer's name and how urgent the request is. You get:

"The customer is Sara Al-Otaibi and this seems quite urgent."

"Name: Sara Al-Otaibi
Urgency: High"

"Sure! Here's what I found — the sender appears to be Sara..."

Three runs, three shapes. Now write the `if` statement that routes the urgent ones.

Regex over model output is a bug you have not hit yet. The fix is not a better parser — it is a **contract**.

PRESENTER NOTES

Everybody in this room who has tried to build on top of a model has written a fragile parser like this, and watched it break in production on a Thursday. The instinct is to write a smarter regex. That is the wrong move — you are patching a symptom of an unconstrained interface. What we want is a contract the model has to satisfy before the response even reaches our code.

THE CEILING · CODE

Ask for JSON, and define the shape

```
from google import genai
from google.genai import types

client = genai.Client(api_key=API_KEY)

schema = {
    "type": "object",
    "properties": {
        "name": {"type": "string"},
        "intent": {"type": "string",
                  "enum": ["question", "complaint", "request"]},
        "urgency": {"type": "string",
                   "enum": ["low", "medium", "high"]},
    },
    "required": ["name", "intent", "urgency"],
}

resp = client.models.generate_content(
    model="gemini-2.5-flash-lite",
    contents=message,
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        response_schema=schema,
        temperature=0,
    ),
)
```

Argument by argument

- `enum` — the model cannot return "quite urgent". Only these three strings.
- `required` — these keys will be present.
- `response_mime_type` — no prose, no markdown fence.
- `response_schema` — the contract itself.
- `temperature=0` — extraction, so no creativity wanted.

PRESENTER NOTES

Walk it top to bottom. The schema is ordinary JSON Schema — the same thing you would write for an API, which is the point.

The two highlighted lines are what actually change the behaviour: one says the response must be JSON, the other says which JSON.

Look at the enum on urgency. That single line is the difference between branching on three known values and parsing English adjectives forever.

The SDK also accepts a Pydantic model in place of that dict, which is what I would use in a real project.

THE CEILING · CODE

This is the moment

```
import json

data = json.loads(resp.text)  # a real dict now

if data["urgency"] == "high":
    page_duty_officer(data)

elif data["intent"] == "complaint":
    open_ticket(data, queue="complaints")

else:
    reply_from_template(data)

# Token counts for every call - this is your bill.
u = resp.usage_metadata
print(u.prompt_token_count, u.candidates_token_count)
```

Why this matters

No regex. No string matching. No "if the answer contains the word urgent".

The model is now a **function that returns a typed value**, and the rest of your system does not need to know an LLM was involved.

Everything on Tuesday — tools, agents — is this idea again, in a loop.

PRESENTER NOTES

Stop here for a second, because this is the slide the whole day was built towards.

That if-statement is ordinary Python. It does not know or care that a language model produced the dictionary it is branching on.

When you can do this, generative AI stops being a chat window and becomes a component you can put inside a system, with everything that follows — testing, monitoring, and being on call for it.

THE CEILING · CODE

Wrap it once, use it all week

```
import json, time

MODEL = "gemini-2.5-flash-lite"

def ask(prompt, system=None, temperature=0.2,
        schema=None, tries=3):
    # Build config; only ask for JSON if a schema was given.
    opts = {"system_instruction": system,
            "temperature": temperature}
    if schema:
        opts["response_mime_type"] = "application/json"
        opts["response_schema"] = schema
    cfg = types.GenerateContentConfig(**opts)

    for n in range(tries):
        try:
            r = client.models.generate_content(
                model=MODEL, contents=prompt, config=cfg)
            return json.loads(r.text) if schema else r.text
        except Exception:
            if n == tries - 1: raise
            time.sleep(2 ** n)  # 1s, 2s, 4s
```

What it buys you

- One place to change the model name.
- Retries with backoff — you *will* hit rate limits tomorrow.
- Schema optional: prose when you want prose, typed data when you don't.

Given complete in the notebook. Read it, don't retype it.

PRESENTER NOTES

You are given this one complete, because the lesson is in reading it, not in typing it.

Catching bare Exception is deliberately blunt for teaching; on Wednesday we replace it with a proper retry decorator that only retries the errors worth retrying.

The sleep line doubles each attempt — one second, two, four. That is exponential backoff, and it is what keeps you from making a rate limit worse.

ZOOM OUT

The model is not the solution

Application What the user actually touches. Streaming, citations, honest failure states.

Orchestration Prompts, chains, the agent loop, retries. Where your logic lives.

Model The generative model itself. Swappable — and it should be.

Tools Functions, APIs, search. How it reaches the world. Tuesday.

Data Your documents, chunks, embeddings, vector store. Monday.

Governance Guardrails, logging, audit, privacy, evaluation. Thursday.

You spend maybe 10% of your time on the model layer. The other five are the job.

PRESENTER NOTES

When somebody says "we are building an AI solution", ask which of these six layers they have actually designed. Usually the answer is the middle one only.

Notice the model sits in the middle and is the easiest layer to replace. The expensive parts are your data and your governance.

Every one of these layers has a day of this course attached to it, and by Thursday you will have touched all six.

HOW THE WORK GOES

The solution lifecycle

01 · FRAME

The use case, the user, and what "good" means before you write anything.

02 · CHOOSE

Prompting, RAG, tools or fine-tuning. Simplest thing that could work.

03 · PROTOTYPE

One notebook, one happy path, real data. Days, not months.

04 · EVALUATE

A golden set and a number. "It looked right" is not a result.

05 · HARDEN

Retries, caching, guardrails, cost and latency you have measured.

06 · OPERATE

Log, trace, watch, re-evaluate. It drifts. So do your documents.

Most teams skip step 4, and then cannot tell whether step 5 helped.

PRESENTER NOTES

Your project this week walks steps one to five, and Thursday morning is a compressed version of step six.

Step four is the one people skip, and it is why so many pilots never reach production — nobody can prove the thing works.

Tomorrow afternoon you will build the golden set that makes step four possible, on your own documents.

LAB · NOTEBOOK 1

Now you build it

day1_first_calls.ipynb

- Your first call, and the token counts.
- Arabic vs English on the same sentence.
- Temperature at 0.0 and at 1.2, three runs each.
- System instructions — two personas, one prompt.
- JSON mode, a schema, and branching on the result.
- The `ask()` wrapper, given complete.

Before you start

- Get your own key at **aistudio.google.com**.
Never share one.
- Put it in Colab **Secrets** (the  panel) as `GEMINI_API_KEY`.
- Free tier: no real SDAIA data, no personal data.
It may train the model.

[Open the notebooks →](#)

PRESENTER NOTES

Everyone creates their own key. If we share one, the whole room shares one rate limit, and the first person to write a loop throttles everybody else.

The most common failure in the next ten minutes is the key not being in Colab secrets, so I will put the key panel on the screen before you start.

Take the privacy warning seriously — this is a free tier, so nothing confidential goes into it. We come back to that on Thursday.

DAY 1

What today was

- 01 Tokens are the unit of everything** — cost, limits, speed. And Arabic costs two to three times more.
- 02 It predicts likely text, not true text** — so fluent and wrong arrive together, by design.
- 03 Prompting is the cheapest improvement available** — role, context, task, format, examples.
- 04 A schema turns output into a contract** — and that is what makes it a component instead of a chat.

Tomorrow: the model still does not know *your* documents. We fix that, and then we measure it.

PRESENTER NOTES

Four things. If you remember nothing else from today, remember that the schema is what made the output safe to build on.

Tomorrow is the longest and most important day of the week, and it starts by asking you three questions from this list, so have them ready.



BEFORE 9:15 TOMORROW

Two things tonight

1 • A GitHub account — you will push your project to it on Thursday.

2 • Pick your documents — 10 to 30 pages you actually care about. A policy, a manual, a handbook. Yours, not a demo set.

Everything you build for the rest of the week runs on that document set. Choose something you would genuinely like to be able to ask questions of.

PRESENTER NOTES

Please do pick real documents. The people who bring a document set they care about are the people who are still working on this in three weeks' time. Nothing confidential though — remember it is going through a free tier. A public policy or a manual is ideal. See you at nine. We start at nine fifteen.